

AIML SESSION 09 ASSIGNMENT SUBMISSION

Name of Student : Vaibhav Vijay Kate

Domain/Subject : AIML (Artificial Intelligence & Machine Learning)

College : Government Polytechnic College Washim

Branch : Information Technology

Session : 09

Q1

Q1. Creating Dictionaries)

Write a program to create dictionaries using different methods:

- a) An empty dictionary using two different ways.
- b) A dictionary with string keys (name, city, course).
- c) A dictionary with integer keys.
- d) A mixed data type dictionary (string, int, float).

Print each dictionary and its type. Add comments explaining the syntax.

Program Code:

a) Empty dictionary using two different ways

```
# Method 1: Using {}
d1 = {}
print("d1 =", d1)
print("Type:", type(d1))
```

```
# Method 2: Using dict()
d2 = dict()
print("d2 =", d2)
print("Type:", type(d2))
```

b) Dictionary with string keys

```
d3 = {
    "name": "Vaibhav",
    "city": "Washim",
    "course": "AI-ML"
}
print("\nd3 =", d3)
print("Type:", type(d3))
```

c) Dictionary with integer keys

```
d4 = {
    1: "India",
    2: "China",
    3: "America",
    4: "Japan"
}
print("\nd4 =", d4)
```

```
print("Type:", type(d4))
```

d) Mixed data type dictionary

```
d5 = {  
    "Brand": "TATA", # String  
    "Wheels": 6,     # Integer  
    "Distance": 129.0 # Float  
}  
print("\nd5 =", d5)  
print("Type:", type(d5))
```

Output Screenshot:



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  
● > & C:/Users/sdf/AppData/Local/Programs/Python/Python314/python.exe "c:/Users/sdf/Desktop/ITR2026/Tasks/Session 09/Q1.py"  
d1 = {}  
Type: <class 'dict'>  
d2 = {}  
Type: <class 'dict'>  
  
d3 = {'name': 'Vaibhav', 'city': 'Washim', 'course': 'AI-ML'}  
Type: <class 'dict'>  
  
d4 = {1: 'India', 2: 'China', 3: 'America', 4: 'Japan'}  
Type: <class 'dict'>  
  
d5 = {'Brand': 'TATA', 'Wheels': 6, 'Distance': 129.0}  
Type: <class 'dict'>  
○ >
```

Q2

Q2. Accessing and Modifying Elements)

Create the dictionary:

```
student = {  
    "name": "YourName",  
    "age": 00,  
    "city": "YourCity",  
    "marks": 00  
}
```

Print the value of "name" using square brackets.

Update the "marks" to 92.

Add a new key "grade" with value "A".

Print the updated dictionary.

Program Code:

```
# Q2. (Accessing and Modifying Elements)
```

```
# Create the dictionary
```

```
student = {  
    "name": "Vaibhav",  
    "marks": 85,  
    "city": "Washim"  
}
```

```
# Print the value of "name" using square brackets
```

```
print("Name:", student["name"])
```

```
# Update the "marks" to 92
```

```
student["marks"] = 92
```

```
# Add a new key "grade" with value "A"
```

```
student["grade"] = "A"
```

```
# Print the updated dictionary
```

```
print("Updated Dictionary:", student)
```

Output Screenshot:



The screenshot shows a terminal window with a dark background. At the top, there are tabs labeled 'PROBLEMS', 'OUTPUT', 'DEBUG CONSOLE', 'TERMINAL' (which is selected and underlined), and 'PORTS'. Below the tabs, a command prompt shows the execution of a Python script. The command is: `> & C:/Users/sdf/AppData/Local/Programs/Python/Python314/python.exe "c:/Users/sdf/Desktop/ITR2026/Tasks/Session 09/Q2.py"`. The output of the script is displayed below the command: `Name: Vaibhav` and `Updated Dictionary: {'name': 'Vaibhav', 'marks': 92, 'city': 'Washim', 'grade': 'A'}`. The prompt `>` is visible at the end of the output line.

```
> & C:/Users/sdf/AppData/Local/Programs/Python/Python314/python.exe "c:/Users/sdf/Desktop/ITR2026/Tasks/Session 09/Q2.py"
Name: Vaibhav
Updated Dictionary: {'name': 'Vaibhav', 'marks': 92, 'city': 'Washim', 'grade': 'A'}
>
```

Q3

Q3. (get(), keys(), values(), items())

Create a dictionary:

```
person = {"name": "Priya", "age": 21, "profession": "Engineer"}
```

Demonstrate:

- a) Using get() to access "age" and a non-existing key "salary" (with default value).
- b) Print all keys using keys().

Session 09 (AIML) - Assignment Questions 1

- c) Print all values using values().
- d) Print all key-value pairs using items().

Program Code:

```
person= {"name": "Priya", "age": 21, "profession": "Engineer"}

# a) Using get() to access "age" and a non-existing key "salary"
print(person.get("age"))
print(person.get("salary", "Not Found"))

# b) Print all keys
print(person.keys())

# c) Print all values
print(person.values())

# d) Print all key-value pairs
print(person.items())
```

Output Screenshot:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
> & C:/Users/sdf/AppData/Local/Programs/Python/Python314/python.exe "c:/Users/sdf/Desktop/ITR2026/Tasks/Session 09/Q3.py"
21
Not Found
dict_keys(['name', 'age', 'profession'])
dict_values(['Priya', 21, 'Engineer'])
dict_items([('name', 'Priya'), ('age', 21), ('profession', 'Engineer')])
>
```

Q4

Q4. Nested Dictionary)

Create a nested dictionary to store information of two students:

Python

```
students = {  
    "s1": {"name": "Rahul", "age": 20, "marks": 88},  
    "s2": {"name": "Sneha", "age": 21, "marks": 95}  
}
```

Print the details of the first student.

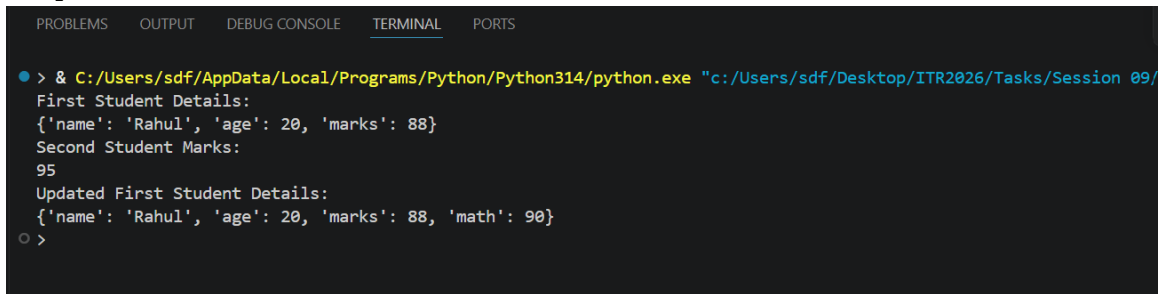
Print the marks of the second student.

Add a new subject "math" with marks 90 for the first student.

Program Code:

```
students= {  
    "s1": {"name": "Rahul", "age": 20, "marks": 88},  
    "s2": {"name": "Sneha", "age": 21, "marks": 95}  
}  
  
# Print the details of the first student  
print("First Student Details:")  
print(students["s1"])  
  
# Print the marks of the second student  
print("Second Student Marks:")  
print(students["s2"]["marks"])  
  
# Add a new subject "math" with marks 90 for the first student  
students["s1"]["math"] = 90  
  
# Print updated first student details  
print("Updated First Student Details:")  
print(students["s1"])
```

Output Screenshot:



The screenshot shows a terminal window with a dark background and light-colored text. At the top, there is a tab bar with five tabs: 'PROBLEMS', 'OUTPUT', 'DEBUG CONSOLE', 'TERMINAL' (which is selected and underlined), and 'PORTS'. Below the tab bar, the terminal displays the output of a Python script. The output starts with a command prompt character '>' followed by a command to run a Python script. The script's output is as follows:

```
> & C:/Users/sdf/AppData/Local/Programs/Python/Python314/python.exe "c:/Users/sdf/Desktop/ITR2026/Tasks/Session 09/  
First Student Details:  
{'name': 'Rahul', 'age': 20, 'marks': 88}  
Second Student Marks:  
95  
Updated First Student Details:  
{'name': 'Rahul', 'age': 20, 'marks': 88, 'math': 90}  
○ >
```


Q5

Q5. (update() and pop())

Create a dictionary: info = {"brand": "Samsung", "model": "A52", "price": 25000}

Update it with new information: {"color": "Black", "price": 24000}.

Remove the key "model" using pop() and print the removed value.

Print the final dictionary.

Program Code:

```
# Q5. (update() and pop())
```

```
# Create a dictionary: info = {"brand": "Samsung", "model": "A52", "price": 25000}
```

```
# Update it with new information: {"color": "Black", "price": 24000}.
```

```
# Remove the key "model" using pop() and print the removed value.
```

```
# Print the final dictionary.
```

```
# Q5. (update() and pop())
```

```
info = {  
    "brand": "Samsung",  
    "model": "A52",  
    "price": 25000  
}
```

```
# Update dictionary
```

```
info.update({  
    "color": "Black",  
    "price": 24000  
})
```

```
# Remove "model" and store removed value
```

```
rm_value = info.pop("model")
```

```
# Print removed value
```

```
print("Removed Value:", rm_value)
```

```
# Print final dictionary
```

```
print("Final Dictionary:", info)
```

Output Screenshot:



The screenshot shows a VS Code terminal window with the 'TERMINAL' tab selected. The terminal displays the output of a Python script executed from the command line. The output shows that the value 'A52' was removed from a dictionary, resulting in a final dictionary with three items: 'brand' (Samsung), 'price' (24000), and 'color' (Black).

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

● > & C:/Users/sdf/AppData/Local/Programs/Python/Python314/python.exe "c:/Users/sdf/Desktop/ITR2026/Tasks/Session 09/Q5.py"
Removed Value: A52
Final Dictionary: {'brand': 'Samsung', 'price': 24000, 'color': 'Black'}
○ >
```

Q6

Q6. (popitem() and clear())

Create a dictionary with at least 5 key-value pairs.

Use popitem() twice and print the removed items each time.

Clear the entire dictionary using clear().

Print the final result.

Add comments explaining the difference between pop() and popitem().

Program Code:

```
# Create a dictionary with 5 key-value pairs
student = {
    "name": "Rahul",
    "age": 20,
    "marks": 88,
    "grade": "A",
    "city": "Mumbai"
}

# Remove and print the last inserted item
removed_item1 = student.popitem()
print("First removed item:", removed_item1)

# Remove and print the next last inserted item
removed_item2 = student.popitem()
print("Second removed item:", removed_item2)

# Print dictionary after popitem()
print("Dictionary after popitem():", student)

# Clear the entire dictionary
student.clear()

# Print final result
print("Final Dictionary:", student)

# Difference between pop() and popitem():
# pop(key) -> Removes a specific key and returns its value.
```

popitem() -> Removes the last inserted key-value pair

Output Screenshot:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python De
> & C:/Users/sdf/AppData/Local/Programs/Python/Python314/python.exe "c:/Users/sdf/Desktop/ITR2026/Tasks/Session 09/Q6.py"
First removed item: ('city', 'Mumbai')
Second removed item: ('grade', 'A')
Dictionary after popitem(): {'name': 'Rahul', 'age': 20, 'marks': 88}
Final Dictionary: {}
>
```

Q7

Q7. (copy() and setdefault())

Create a dictionary: d = {"a": 1, "b": 2}

Make a copy of it.

Session 09 (AIML) - Assignment Questions 2

Use setdefault() to add key "c" with value 3 (if it doesn't exist).

Use setdefault() again for an existing key "a".

Print the original and copied dictionaries.

Program Code:

```
# Q7. (copy() and setdefault())
```

```
# Create a dictionary
```

```
d = {"a": 1, "b": 2}
```

```
# Make a copy of the dictionary
```

```
d_copy = d.copy()
```

```
# Add key "c" with value 3 if it doesn't exist
```

```
d.setdefault("c", 3)
```

```
# Use setdefault() on existing key "a"
```

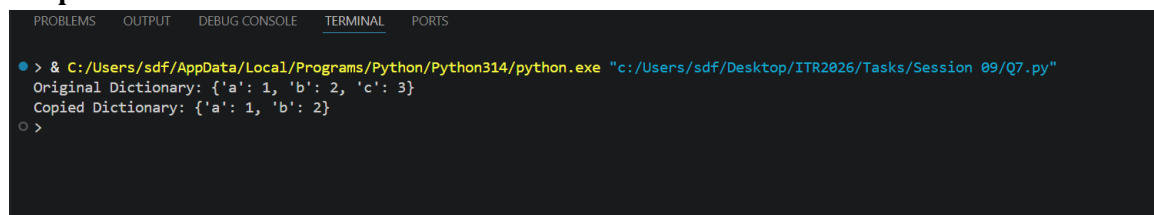
```
d.setdefault("a", 100)
```

```
# Print original and copied dictionaries
```

```
print("Original Dictionary:", d)
```

```
print("Copied Dictionary:", d_copy)
```

Output Screenshot:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
> & C:/Users/sdf/AppData/Local/Programs/Python/Python314/python.exe "c:/Users/sdf/Desktop/ITR2026/Tasks/Session 09/Q7.py"
Original Dictionary: {'a': 1, 'b': 2, 'c': 3}
Copied Dictionary: {'a': 1, 'b': 2}
>
```

Q8

Q8. (fromkeys() and Membership)

Write a program that:

Uses dict.fromkeys() to create a dictionary with keys ["name", "age", "city"] and default value None.

Takes user input to fill these keys.

Checks if a given key exists using in operator.

Program Code:

```
# Q8. (fromkeys() and Membership)

# Create a dictionary with default value None
keys = ["name", "age", "city"]
person = dict.fromkeys(keys, None)

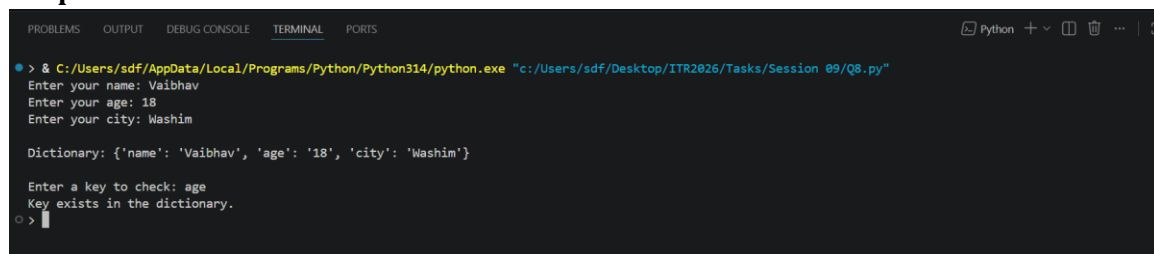
# Take user input
person["name"] = input("Enter your name: ")
person["age"] = input("Enter your age: ")
person["city"] = input("Enter your city: ")

# Print dictionary
print("\nDictionary:", person)

# Check if a key exists
key = input("\nEnter a key to check: ")

if key in person:
    print("Key exists in the dictionary.")
else:
    print("Key does not exist in the Dictionary.")
```

Output Screenshot:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python + - [ ] [ ] [ ] [ ] [ ]
> & C:/Users/sdf/AppData/Local/Programs/Python/Python314/python.exe "c:/Users/sdf/Desktop/ITR2026/Tasks/Session 09/Q8.py"
Enter your name: Vaibhav
Enter your age: 18
Enter your city: Washim

Dictionary: {'name': 'Vaibhav', 'age': '18', 'city': 'Washim'}

Enter a key to check: age
Key exists in the dictionary.
>
```

Q9

Q9. Practical Application)

Write a program to store and manage contact information using a dictionary:

Take name, phone number, and email as input.

Store them in a dictionary with name as key.

Allow the user to search a contact by name using get().

Print all contacts using items().

Program Code:

```
# Contact Management System

contacts = {}

# Take input from user
name = input("Enter Name: ")
phone = input("Enter Phone Number: ")
email = input("Enter Email: ")

# Store contact information
contacts[name] = {
    "phone": phone,
    "email": email
}

# Search contact using get()
search_name = input("\nEnter name to search: ")

contact = contacts.get(search_name)

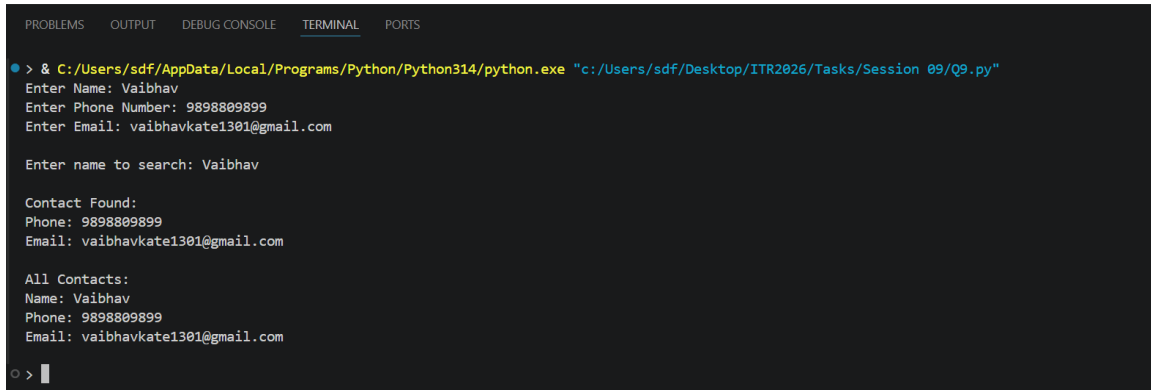
if contact:
    print("\nContact Found:")
    print("Phone:", contact["phone"])
    print("Email:", contact["email"])
else:
    print("Contact not found.")

# Print all contacts using items()
print("\nAll Contacts:")

for name, details in contacts.items():
```

```
print("Name:", name)
print("Phone:", details["phone"])
print("Email:", details["email"])
print()
```

Output Screenshot:



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

> & C:/Users/sdf/AppData/Local/Programs/Python/Python314/python.exe "c:/Users/sdf/Desktop/ITR2026/Tasks/Session 09/Q9.py"
Enter Name: Vaibhav
Enter Phone Number: 9898809899
Enter Email: vaibhavkate1301@gmail.com

Enter name to search: Vaibhav

Contact Found:
Phone: 9898809899
Email: vaibhavkate1301@gmail.com

All Contacts:
Name: Vaibhav
Phone: 9898809899
Email: vaibhavkate1301@gmail.com

○ > |
```


Q10

Q10. Mini Project - Combined Concepts)

Create a Student Management System using a dictionary:

Repeatedly ask the user to enter student details (roll number as key, and a nested dictionary containing name, age, and marks).

Use a while loop with a menu:

Add new student

Update marks of a student

Search student by roll number

Display all students

Remove a student

Exit

Session 09 (AIML) - Assignment Questions 3

Use appropriate dictionary methods (get(), update(), pop(), etc.). Add meaningful comments.

Session 09 (AIML) - Assignment Questions 4

Program Code:

```
# Student Management System
```

```
students = {}
```

```
while True:
```

```
    print("\n===== Student Management System =====")
```

```
    print("1. Add New Student")
```

```
    print("2. Update Marks")
```

```
    print("3. Search Student")
```

```
    print("4. Display All Students")
```

```
    print("5. Remove Student")
```

```
    print("6. Exit")
```

```
    choice = input("Enter your choice (1-6): ")
```

```
    # 1. Add New Student
```

```
    if choice == "1":
```

```
        roll = input("Enter Roll Number: ")
```

```
        name = input("Enter Name: ")
```

```
        age = int(input("Enter Age: "))
```

```
        marks = float(input("Enter Marks: "))
```

```
students[roll] = {  
    "name": name,  
    "age": age,  
    "marks": marks  
}
```

```
print("Student added successfully!")
```

```
# 2. Update Marks
```

```
elif choice == "2":
```

```
    roll = input("Enter Roll Number: ")
```

```
    if roll in students:
```

```
        new_marks = float(input("Enter New Marks: "))
```

```
        students[roll]["marks"] = new_marks
```

```
        print("Marks updated successfully!")
```

```
    else:
```

```
        print("Student not found!")
```

```
# 3. Search Student
```

```
elif choice == "3":
```

```
    roll = input("Enter Roll Number: ")
```

```
    student = students.get(roll)
```

```
    if student:
```

```
        print("\nStudent Details:")
```

```
        print("Name:", student["name"])
```

```
        print("Age:", student["age"])
```

```
        print("Marks:", student["marks"])
```

```
    else:
```

```
        print("Student not found!")
```

```
# 4. Display All Students
```

```
elif choice == "4":
```

```
    if students:
```

```
        print("\nAll Students:")
```

```
        for roll, details in students.items():
```

```
            print(f"\nRoll No: {roll}")
```

```
            print("Name:", details["name"])
```

```
            print("Age:", details["age"])
```

```
            print("Marks:", details["marks"])
```

```
    else:
```

```

        print("No students available!")

# 5. Remove Student
elif choice == "5":
    roll = input("Enter Roll Number: ")

    if roll in students:
        removed_student = students.pop(roll)
        print("Removed Student:", removed_student)
    else:
        print("Student not found!")

# 6. Exit
elif choice == "6":
    print("Exiting Student Management System...")
    break

else:
    print("Invalid choice! Please enter a number between 1 and 6.")

```

Output Screenshot:

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
○ > & C:/Users/sdf/AppData/Local/Programs/Python/Python314/python.exe "c:/Users/sdf/Desktop/ITR2026/Tasks/Session 09/Q10.py"

===== Student Management System =====
1. Add New Student
2. Update Marks
3. Search Student
4. Display All Students
5. Remove Student
6. Exit
Enter your choice (1-6): 1
Enter Roll Number: 530
Enter Name: Vaibhav Kate
Enter Age: 18
Enter Marks: 89
Student added successfully!

===== Student Management System =====
1. Add New Student
2. Update Marks
3. Search Student
4. Display All Students
5. Remove Student
6. Exit
Enter your choice (1-6): █

```